

# Machine Learning

PHYS 453

Dr. Daugherty



Part 0

# PYTORCH OVERVIEW

## 1.17. Neural network models (supervised)

**Warning:** This implementation is not intended for large-scale applications. In particular, scikit-learn offers no GPU support. For much faster, GPU-based implementations, as well as frameworks offering much more flexibility to build deep learning architectures, see [Related Projects](#).

Instead use <https://pytorch.org/>

Optionally add:

- <https://github.com/skorch-dev/skorch> - brings pytorch into sklearn
- lightning / ignite / keras – higher level APIs meant to streamline work

Advantages

- Designed to run on GPU farms
- Flexible deep NNs with ever-growing list of tools

# pytorch

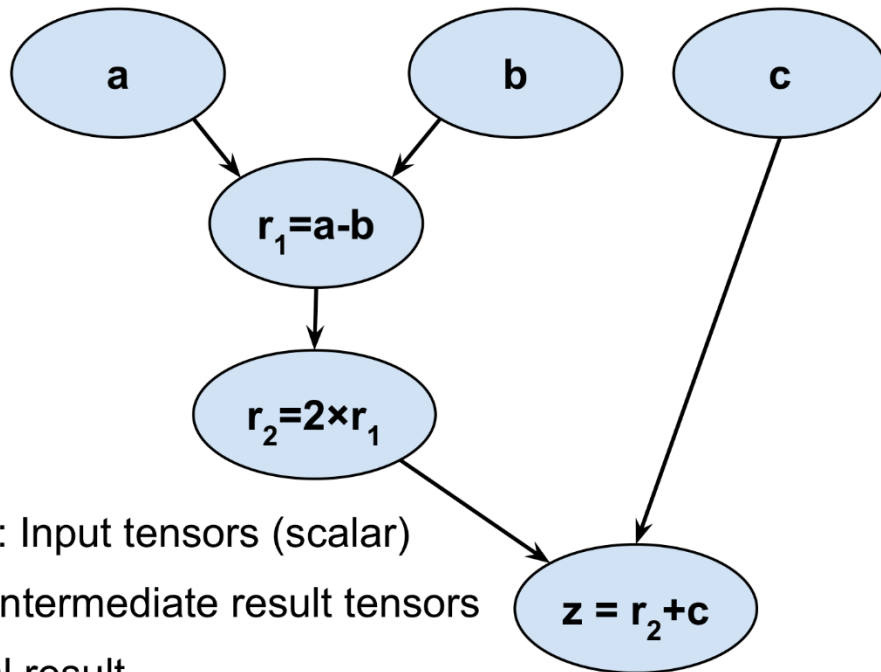
The most popular and powerful tool by far for advanced deep learning is pytorch

- wish I had a second semester to do this...
- tedious and painful to get started, lots and lots of lines of “boilerplate” needed to do anything
- some tools exist to make it easier, but you may be better off just doing straight pytorch
  - skorch: brings pytorch into sklearn <https://github.com/skorch-dev/skorch>
  - lightning, ignite, keras

# Autograd

- pytorch defines its own data types called **tensors** that are like numpy arrays except that they automatically keep track of all computation steps to calculate partial derivatives
- designed around efficient **gradient descent**

Computation graph implementing  
the equation  $z = 2 \times (a - b) + c$



```
: w = torch.tensor(1.0, requires_grad=True)
  b = torch.tensor(0.5, requires_grad=True)

  x = torch.tensor([1.4])
  y = torch.tensor([2.1])

  z = torch.add(torch.mul(w, x), b)

  loss = (y - z).pow(2).sum()
  loss.backward()

  print('dL/dw : ', w.grad)
  print('dL/db : ', b.grad)
```

```
dL/dw :  tensor(-0.5600)
dL/db :  tensor(-0.4000)
```

```
: # verifying the computed gradient dL/dw
  print(2 * x * ((w * x + b) - y))
```

```
tensor([-0.5600], grad_fn=<MulBackward0>)
```

# pytorch

- Pytorch is powerful, which is synonym for complicated
- Good news: you must custom-define everything
- Bad news: you must custom-define everything

Let's talk about 3 main components

1. dataloader
2. model
3. training

# Data Loader

- [https://docs.pytorch.org/tutorials/beginner/basics/data\\_tutorial.html](https://docs.pytorch.org/tutorials/beginner/basics/data_tutorial.html)
- designed to be parallel processed and GPU/TPU friendly
- everything is **batched** = how many training samples to process at once
- handles transforms like our pipelines do
- some pre-defined dataloaders exist for common datasets

Code for processing data samples can get messy and hard to maintain; we ideally want our dataset code to be decoupled from our model training code for better readability and modularity. PyTorch provides two data primitives:

`torch.utils.data.DataLoader` and `torch.utils.data.Dataset` that allow you to use pre-loaded datasets as well as your own data. `Dataset` stores the samples and their corresponding labels, and `DataLoader` wraps an iterable around the `Dataset` to enable easy access to the samples.

# Model

- [https://docs.pytorch.org/tutorials/beginner/basics/buildmodel\\_tutorial.html](https://docs.pytorch.org/tutorials/beginner/basics/buildmodel_tutorial.html)
- Define a custom function for how your model calculates output and what the adjustable parameters are
- use Sequential layers to make it easier

```
class NeuralNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()
        self.linear_relu_stack = nn.Sequential(
            nn.Linear(28*28, 512),
            nn.ReLU(),
            nn.Linear(512, 512),
            nn.ReLU(),
            nn.Linear(512, 10),
        )

    def forward(self, x):
        x = self.flatten(x)
        logits = self.linear_relu_stack(x)
        return logits
```



See the complete list of layers at: <https://docs.pytorch.org/docs/stable/nn.html>

## **torch.nn**

[Containers](#)

[Convolution Layers](#)

[Pooling layers](#)

[Padding Layers](#)

[Non-linear Activations \(weighted sum, nonlinearity\)](#)

[Non-linear Activations \(other\)](#)

[Normalization Layers](#)

[Recurrent Layers](#)

[Transformer Layers](#)

[Linear Layers](#)

[Dropout Layers](#)

[Sparse Layers](#)

[Distance Functions](#)

[Loss Functions](#)

[Vision Layers](#)

[Shuffle Layers](#)

[DataParallel Layers \(multi-GPU, distributed\)](#)

[Utilities](#)

[Quantized Functions](#)

[Lazy Modules Initialization](#)

Just a few to know for now:

- linear – basic fully connected NN layer
- activation functions – ReLu, tanh, sigmoid
- softmax
- Dropout – regularization by randomly setting weights to zero for one epoch
- BatchNorm

Some we will use soon for images

- Conv2D
- MaxPool2D

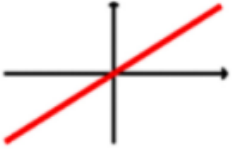
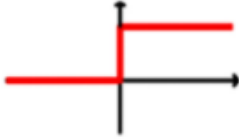
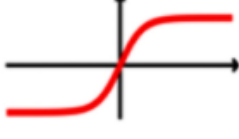
| Activation function               | Equation                                                                                                                                            | Example                            | 1D graph                                                                             |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------|--------------------------------------------------------------------------------------|
| Linear                            | $\sigma(z) = z$                                                                                                                                     | Adaline, linear regression         |    |
| Unit step<br>(Heaviside function) | $\sigma(z) = \begin{cases} 0 & z < 0 \\ 0.5 & z = 0 \\ 1 & z > 0 \end{cases}$                                                                       | Perceptron variant                 |    |
| Sign<br>(signum)                  | $\sigma(z) = \begin{cases} -1 & z < 0 \\ 0 & z = 0 \\ 1 & z > 0 \end{cases}$                                                                        | Perceptron variant                 |    |
| Piece-wise linear                 | $\sigma(z) = \begin{cases} 0 & z \leq -\frac{1}{2} \\ z + \frac{1}{2} & -\frac{1}{2} \leq z \leq \frac{1}{2} \\ 1 & z \geq \frac{1}{2} \end{cases}$ | Support vector machine             |    |
| Logistic<br>(sigmoid)             | $\sigma(z) = \frac{1}{1 + e^{-z}}$                                                                                                                  | Logistic regression, multilayer NN |    |
| Hyperbolic tangent<br>(tanh)      | $\sigma(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$                                                                                                     | Multilayer NN, RNNs                |  |
| ReLU                              | $\sigma(z) = \begin{cases} 0 & z < 0 \\ z & z > 0 \end{cases}$                                                                                      | Multilayer NN, CNNs                |  |

Figure 12.11: The activation functions covered in this book

# Training

- [https://docs.pytorch.org/tutorials/beginner/basics/optimization\\_tutorial.html](https://docs.pytorch.org/tutorials/beginner/basics/optimization_tutorial.html)
- You define a custom loss function and optimizer, then define a loop that calculates the gradients and adjusts parameters

```
def train_loop(dataloader, model, loss_fn, optimizer):
    size = len(dataloader.dataset)
    # Set the model to training mode - important for batch normalization and dropout layers
    # Unnecessary in this situation but added for best practices
    model.train()
    for batch, (X, y) in enumerate(dataloader):
        # Compute prediction and loss
        pred = model(X)
        loss = loss_fn(pred, y)

        # Backpropagation
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()

    if batch % 100 == 0:
        loss, current = loss.item(), batch * batch_size + len(X)
        print(f"loss: {loss:>7f} [{current:>5d}/{size:>5d}]")
```

```
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)

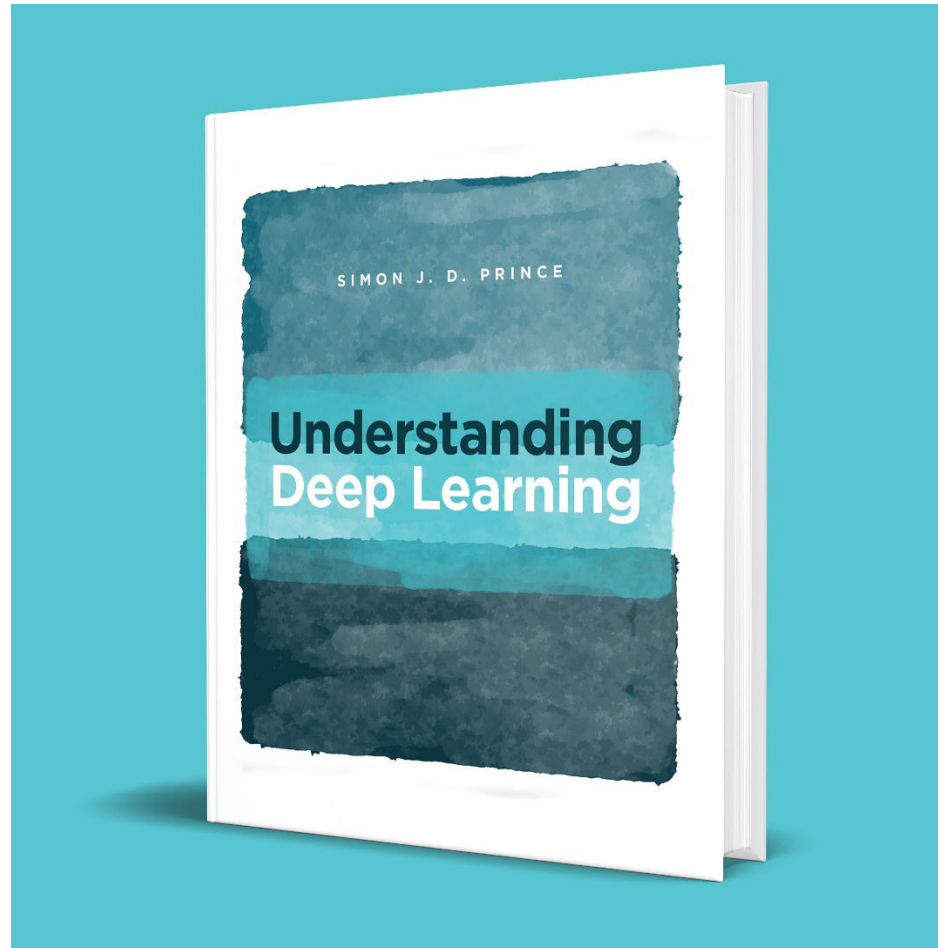
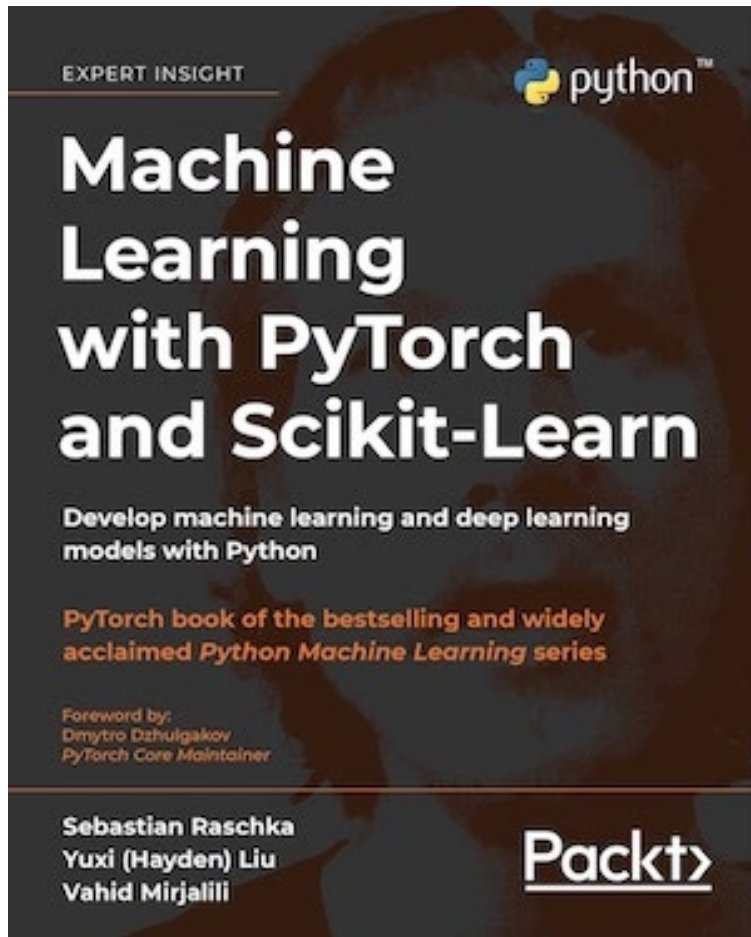
epochs = 10
for t in range(epochs):
    print(f"Epoch {t+1}\n-----")
    train_loop(train_dataloader, model, loss_fn, optimizer)
    test_loop(test_dataloader, model, loss_fn)
print("Done!")
```

Part 1

# **SIMPLE EXAMPLES**

# Sources

- <https://docs.pytorch.org/tutorials/intro.html>
- <https://github.com/rasbt/machine-learning-book/tree/main>
- <https://udlbook.github.io/udlbook/>



[https://github.com/rasbt/machine-learning-book/blob/main/ch12/ch12\\_part2.ipynb](https://github.com/rasbt/machine-learning-book/blob/main/ch12/ch12_part2.ipynb)

```
from torch.utils.data import TensorDataset
from torch.utils.data import DataLoader
```

```
X_train_norm = (X_train - np.mean(X_train)) / np.std(X_train)
X_train_norm = torch.from_numpy(X_train_norm).float()
y_train = torch.from_numpy(y_train)
```

```
train_ds = TensorDataset(X_train_norm, y_train)
```

```
torch.manual_seed(1)
batch_size = 2
train_dl = DataLoader(train_ds, batch_size, shuffle=True)
```

```
class Model(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super().__init__()
        self.layer1 = nn.Linear(input_size, hidden_size)
        self.layer2 = nn.Linear(hidden_size, output_size)
```

```
    def forward(self, x):
        x = self.layer1(x)
        x = nn.Sigmoid()(x)
        x = self.layer2(x)
        x = nn.Softmax(dim=1)(x)
        return x
```

```
input_size = X_train_norm.shape[1]
hidden_size = 16
output_size = 3
```

```
model = Model(input_size, hidden_size, output_size)
```

```
learning_rate = 0.001
```

```
loss_fn = nn.CrossEntropyLoss()
```

```
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
```

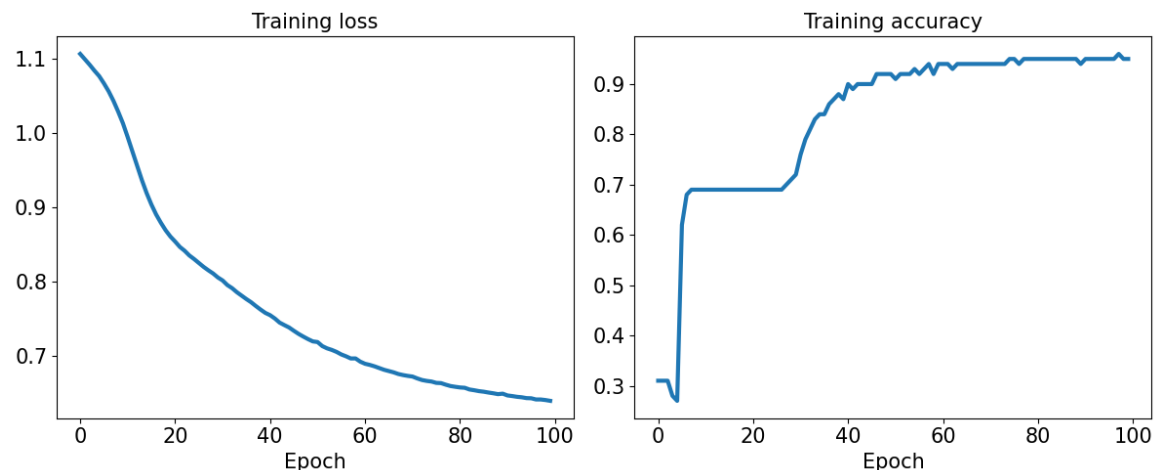
```
num_epochs = 100
loss_hist = [0] * num_epochs
accuracy_hist = [0] * num_epochs
```

```
for epoch in range(num_epochs):
```

```
    for x_batch, y_batch in train_dl:
        pred = model(x_batch)
        loss = loss_fn(pred, y_batch.long())
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()
```

```
    loss_hist[epoch] += loss.item()*y_batch.size(0)
    is_correct = (torch.argmax(pred, dim=1) == y_batch).float()
    accuracy_hist[epoch] += is_correct.sum()
```

```
    loss_hist[epoch] /= len(train_dl.dataset)
    accuracy_hist[epoch] /= len(train_dl.dataset)
```



```
import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets
from torchvision.transforms import ToTensor

training_data = datasets.FashionMNIST(
    root="data",
    train=True,
    download=True,
    transform=ToTensor()
)

test_data = datasets.FashionMNIST(
    root="data",
    train=False,
    download=True,
    transform=ToTensor()
)

train_dataloader = DataLoader(training_data, batch_size=64)
test_dataloader = DataLoader(test_data, batch_size=64)

class NeuralNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()
        self.linear_relu_stack = nn.Sequential(
            nn.Linear(28*28, 512),
            nn.ReLU(),
            nn.Linear(512, 512),
            nn.ReLU(),
            nn.Linear(512, 10),
        )

    def forward(self, x):
        x = self.flatten(x)
        logits = self.linear_relu_stack(x)
        return logits

model = NeuralNetwork()
```

```
def train_loop(dataloader, model, loss_fn, optimizer):
    size = len(dataloader.dataset)
    # Set the model to training mode - important for batch normalization and dropout layers
    # Unnecessary in this situation but added for best practices
    model.train()
    for batch, (X, y) in enumerate(dataloader):
        # Compute prediction and loss
        pred = model(X)
        loss = loss_fn(pred, y)

        # Backpropagation
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()

        if batch % 100 == 0:
            loss, current = loss.item(), batch * batch_size + len(X)
            print(f"loss: {loss:>7f} [{current:>5d}/{size:>5d}]")

def test_loop(dataloader, model, loss_fn):
    # Set the model to evaluation mode - important for batch normalization and dropout layers
    # Unnecessary in this situation but added for best practices
    model.eval()
    size = len(dataloader.dataset)
    num_batches = len(dataloader)
    test_loss, correct = 0, 0

    # Evaluating the model with torch.no_grad() ensures that no gradients are computed during test mode
    # also serves to reduce unnecessary gradient computations and memory usage for tensors with requires_grad=True
    with torch.no_grad():
        for X, y in dataloader:
            pred = model(X)
            test_loss += loss_fn(pred, y).item()
            correct += (pred.argmax(1) == y).type(torch.float).sum().item()

    test_loss /= num_batches
    correct /= size
    print(f"Test Error: \n Accuracy: {(100*correct):>0.1f}%, Avg loss: {test_loss:>8f} \n")
```

```
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)

epochs = 10
for t in range(epochs):
    print(f"Epoch {t+1}\n-----")
    train_loop(train_dataloader, model, loss_fn, optimizer)
    test_loop(test_dataloader, model, loss_fn)
    print("Done!")
```